

Code Review Report: AZANIR/cypressAllure

Repository reviewed: <https://github.com/AZANIR/cypressAllure>

Review date: 2026-07-09

Reviewer: Denys

Scope

The review covered the Cypress test project structure, test code, page objects, fixtures, Cypress configuration, npm scripts, Allure setup, README file, GitHub Actions workflow, TypeScript configuration, and committed generated artifacts.

Execution Check

I installed dependencies with `npm ci`. Installation completed, but npm reported 15 vulnerabilities: 6 moderate, 5 high, and 4 critical.

I also tried to run the tests with `npm run cy:run`. Cypress 10.3.0 failed during the initial smoke test before any spec files started:

Cypress failed to start.

Command failed with exit code 1:

`Cypress.exe --smoke-test`

Important correction: the direct project-level blocker is not only the Cypress version. The config file is TypeScript: `config/config.config.ts`. Cypress needs `ts-node` to load this TypeScript config, but `ts-node` is missing from `package.json`. This can prevent Cypress from parsing the config and starting correctly.

Because of this, the review is based mainly on static code inspection plus the dependency installation result.

Findings and Recommendations

1. Missing `ts-node` blocks Cypress TypeScript config loading

Files: `config/config.config.ts`, `package.json`

The project uses a TypeScript Cypress config file: `config/config.config.ts`. However, `package.json` does not include `ts-node` in `devDependencies`. Cypress needs this dependency to compile and load TypeScript config files.

Recommendation: add `ts-node` to `devDependencies` and verify that `npm run cy:run` can load the config before executing specs.

2. Failing assertion in wrong email format test

File: `cypress/e2e/tests/login.test.ts`, line 35

The expected error message is `Invalid email addresssssss..`. This looks like a typo and will not match the real application message.

Recommendation: replace it with the actual expected text, for example Invalid email address., and preferably store expected messages in fixture/test data instead of hardcoding them in test steps.

3. Fixture data stored on this.data is unsafe and fails strict typing

File: cypress/e2e/tests/login.test.ts, lines 5-9 and 18-34

The tests assign fixture data to this.data and then read it later in test cases. This pattern is fragile in Cypress and becomes a TypeScript blocker under "strict": true, because data is not declared on Mocha's Context type. It can fail compilation before tests even start.

Recommendation: avoid this.data. Load fixture data into a typed variable in the spec scope, or use cy.fixture(...).as(...) with correct typing. If the team keeps Mocha context, extend the context type explicitly.

4. Hardcoded credentials are still used in one test

File: cypress/e2e/tests/login.test.ts, line 12

The first login test uses hardcoded email and password, while the project already has cypress/fixtures/users.json.

Recommendation: use fixture data consistently for all credentials. This will make the tests easier to maintain and avoid duplication.

5. Duplicate positive login coverage

File: cypress/e2e/tests/login.test.ts, lines 11-21

There are two positive login tests with the same flow: login, validate account page, logout, validate logout. The only difference is where the credentials come from.

Recommendation: keep one positive login test that uses fixture data. If a second positive test is needed, it should cover a different condition.

6. Sample test should be removed or replaced with a real account test

File: cypress/e2e/tests/myAccount.test.ts, lines 3-10

The suite named My Account Functionality visits https://google.com, contains a commented login call, and the test only prints to the console.

Recommendation: remove this spec or implement real My Account test cases using the application under test and the Page Object Model.

7. Commented and unused code is left in tests

File: cypress/e2e/tests/myAccount.test.ts, line 6

File: config/config.config.ts, lines 17-18

The project contains commented code and default migration comments.

Recommendation: remove commented-out code and generated comments that do not add value. The repository should contain clean production test code.

8. Page Object Model boundaries are violated

Files: `cypress/e2e/pages/loginPage.ts`, `cypress/e2e/pages/myAccountPage.ts`

The `login()` method in `LoginPage` starts by clicking `this.signInLink`. This couples the login action to a header navigation link instead of representing the login form itself. If a test opens the login page directly, `login()` can fail or perform an unnecessary navigation step.

`validateSuccessfulLogout()` is located in `MyAccountPage`, but it asserts on `loginPage.signInLink`, which belongs to a different page object. This leaks page boundaries and makes page objects depend on each other in a way that is hard to maintain.

Recommendation: keep navigation actions separate from form actions. `LoginPage.login()` should only fill and submit the login form. Logout validation should be moved to the page object that owns the asserted element, or changed to validate a stable post-logout state through a dedicated page object.

9. Cypress custom command for login is missing

File: `cypress/support/commands.js`, lines 1-25

`commands.js` contains only the default Cypress template comments. There is no reusable `cy.login()` command.

Recommendation: add a custom Cypress command for login or remove the unused `commands.js` import. For this project, a `cy.login(email, password)` command would reduce repeated login steps and match Cypress best practices.

10. README is incomplete and has formatting issues

File: `README.md`, lines 1-35

Problems:

- The first heading is broken: `# cypress-e2e-allure# Cypress-e2e-Allure...`
- The clone command has no repository URL.
- The instruction Change "Username" and "Password" does not explain which file should be changed.
- It does not explain all available npm scripts.
- It does not clearly explain how to generate and open the Allure report.
- It uses external screenshots instead of clear project documentation.

Recommendation: rewrite README with sections for prerequisites, installation, test execution, opening Cypress, report generation, project structure, and test data location.

11. Allure result paths are inconsistent

Files:

- config/config.config.ts, line 14
- package.json, lines 10-11
- .github/workflows/cyAllureRun.yml, line 27

The Cypress config sets allureResultsPath to ../allure-results, the report script reads from reports/ui/allure-results, and the allure:generate script reads from allure-results.

Recommendation: use one consistent Allure results directory across local scripts, Cypress config, and CI. For example: reports/ui/allure-results for results and reports/ui/allure-report for the generated report.

12. report script can hang CI

File: package.json, line 10

The report script runs allure open reports/ui/allure-report. allure open starts a persistent local web server and keeps the terminal process alive. If cy:e2e:run is used in CI, the pipeline can hang until the job times out.

Recommendation: split report generation and report opening into separate scripts. CI should run only allure generate. A local-only command can run allure open.

13. Package scripts are not fully cross-platform

File: package.json, lines 7-12

The clear script uses `rm -r reports/ || true`, which works on Linux/macOS but can fail on Windows shells. Cross-platform issues can also appear when environment values are passed in npm scripts. For example, `--env allure=true` is supported by Cypress CLI, but the project should still keep environment initialization consistent across Windows, Linux, and CI shells when more variables are added.

Recommendation: use cross-platform utilities in package.json. rimraf can replace shell-specific cleanup, and cross-env should be used when npm scripts need to initialize environment variables in a portable way.

```
"clear": "rimraf reports allure-results allure-report"
```

14. Legacy Plugins Migration is still present

File: config/config.config.ts, lines 19-21

Inside e2e.setupNodeEvents, the project still calls the old Cypress plugins file:

```
return require('cypress/plugins/index.js')(on, config)
```

This is a legacy migration artifact from Cypress versions before 10. Keeping both the modern Cypress config and the old cypress/plugins structure makes the setup harder to understand and maintain.

Recommendation: remove the legacy plugin structure. Move plugin setup, including Allure registration, directly into `setupNodeEvents` in `config.config.ts`, then delete the old `cypress/plugins` folder.

15. Video recording is always enabled

File: `config/config.config.ts`, line 4

The config hardcodes `video: true`. Recording video for every local run slows down execution and uses extra CPU and disk resources.

Recommendation: set `video: false` by default and enable video only for CI runs, for example with `cypress run --config video=true`. If the team mainly needs videos for debugging failures, configure recording only for failed tests where supported by the chosen Cypress/reporting setup.

16. Page load and exec timeouts are too aggressive

File: `config/config.config.ts`, lines 6 and 8

`pageLoadTimeout` is set to 30000, while Cypress default is 60 seconds. This is risky because the configured base URL, `http://automationpractice.com/index.php`, is unstable and returned 502 Bad Gateway during review. The base URL also uses the insecure `http://` protocol.

The same configuration also sets `execTimeout` to 5000, down from the Cypress default of 60000 ms. This is a major CI risk because any Cypress system command can fail on a slow runner or during report/tool setup.

Recommendation: restore the default timeouts or increase them based on real environment performance. Prefer a stable demo environment that works over `https://`.

17. CI workflow uses outdated actions and Node version

File: `.github/workflows/cyAllureRun.yml`, lines 12, 15, 17, 29, 35

The workflow uses Node 14.17.0 and old GitHub Actions versions: `actions/checkout@v2`, `actions/setup-node@v2`, and `actions/upload-artifact@v1`.

Recommendation: update to a supported Node version and newer action versions, for example `checkout@v4`, `setup-node@v4`, and `upload-artifact@v4`.

18. CI workflow does not cache the Cypress binary

File: `.github/workflows/cyAllureRun.yml`

The workflow enables npm cache through `actions/setup-node`, but it does not cache the Cypress binary directory, usually `~/.cache/Cypress`. This forces GitHub Actions to download Cypress again on each run, making CI slower and more fragile.

Recommendation: add caching for the Cypress binary path or use the official Cypress GitHub Action, which handles installation and caching more predictably.

19. GitHub Pages deploy token is configured incorrectly

File: `.github/workflows/cyAllureRun.yml`, line 38

The workflow uses:

```
ACCESS_TOKEN: ${{'secrets.ACCESS_TOKEN'}}
```

This is a string expression, not a real secret reference.

Recommendation: use the correct secret syntax:

```
ACCESS_TOKEN: ${{ secrets.ACCESS_TOKEN }}
```

Also consider deploying reports to a separate gh-pages branch instead of pushing generated reports back to master.

20. npm install is used in CI instead of npm ci

File: `.github/workflows/cyAllureRun.yml`, line 22

The repository has `package-lock.json`, but CI runs `npm i`. This can install dependency versions different from the lockfile expectation.

Recommendation: use `npm ci` in CI for reproducible builds.

21. Generated Allure report is committed to the repository

Folder: `docs/`

The repository contains generated Allure report files, including JSON data and media attachments. This increases repository size and mixes source code with generated output.

Recommendation: keep generated reports as CI artifacts or deploy them to GitHub Pages from a separate branch/folder. Do not mix report output with source code unless the repository intentionally uses `docs/` only for published static content. Also verify `.gitignore` against the actual output paths. `config.config.ts` overrides Cypress folders to `reports/screenshots` and `reports/videos`, so those specific paths should be ignored together with `allure-results` and `allure-report` to prevent local media files and raw report data from filling the Git repository.

22. Test environment URL may be unstable

File: `config/config.config.ts`, line 22

The project uses `http://automationpractice.com/index.php` as `baseUrl`. During review, this URL returned a 502 Bad Gateway response through a fetch check.

Recommendation: use a stable test environment, document it in README, and consider moving `baseUrl` to environment-specific configuration.

23. Page object selectors can be improved

File: cypress/e2e/pages/loginPage.ts, lines 4-9

Selectors are based on CSS classes and text search, for example `.login`, `.alert-danger > ol > li`, and `p:contains("error")`. These selectors can break when the UI layout or text changes.

Recommendation: use stable selectors where possible, such as `data-testid` attributes. If the tested application cannot be changed, centralize selectors and avoid broad text-based selectors.

24. Login error assertion can be flaky for multiple list items

File: cypress/e2e/pages/loginPage.ts, lines 9 and 24

The locator `.alert-danger > ol > li` can return a list of error items. Cypress concatenates text from all matched elements when using `should('have.text', errorMessage)`, so the assertion can fail if the page renders multiple errors.

Recommendation: assert a specific list item, use `should('contain.text', errorMessage)` on the parent alert, or validate the list contents intentionally.

25. Assertions should be more robust

File: cypress/e2e/pages/myAccountPage.ts, line 8

File: cypress/e2e/pages/loginPage.ts, line 24

The tests use exact text assertions with `should('have.text', ...)`. This can fail because of whitespace or small UI text changes.

Recommendation: use `should('contain.text', ...)` when exact full text is not required, or normalize text before assertion.

26. TypeScript compiler configuration needs small cleanup

File: tsconfig.json

The TypeScript config already has `"strict": true`, which is a good decision for Page Object typing and safer test code. The `types` array includes both `node` and `cypress`, which helps Cypress autocomplete. However, the `include` section references dependencies through paths like `../node_modules/...`. Depending on where the compiler is started from, paths that go one level above the project can be fragile.

Recommendation: keep `"strict": true`. Review the `include` section and prefer standard project-local paths where possible, avoiding `../node_modules/...` references unless they are strictly required by the plugin.

27. Dependency versions are old

File: package.json, lines 32-37

The project uses older versions of Cypress, TypeScript, Allure commandline, and the Allure plugin. Dependency installation reported moderate, high, and critical vulnerabilities.

Recommendation: update dependencies step by step, run the test suite after each upgrade, and add a regular dependency audit process.

Positive Notes

- The project already has a Page Object Model structure.
- Test data is partially moved to fixture files.
- Allure reporting is integrated.
- GitHub Actions workflow exists, so CI can be improved instead of created from scratch.
- TypeScript strict mode is enabled, which is a good baseline for safer test code.

Suggested Priority

1. Add ts-node and fix the TypeScript fixture/context issue so Cypress can load the config and compile tests.
2. Remove the sample Google test and fix the failing expected error message.
3. Clean the Cypress configuration from legacy plugin migration code and fix cross-platform package scripts.
4. Split CI-safe report generation from local allure open, then fix Allure paths.
5. Fix Page Object Model boundaries and add a reusable login command.
6. Update CI workflow syntax, actions, Cypress binary caching, and secret usage.
7. Update dependencies and clean generated artifacts from the main source branch.

Conclusion

The project has a good basic structure for a Cypress + TypeScript + Allure framework, but it needs cleanup before it can be considered reliable. The main blockers are missing ts-node for TypeScript config loading, strict TypeScript failures around this.data, inconsistent report configuration, a CI-hanging allure open script, legacy Cypress plugin setup, non-portable scripts, weak Page Object boundaries, unused sample code, duplicated/hardcoded test data, outdated CI/dependencies, and at least one assertion that is likely to fail.